

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Name	Vinay Sathish	
Roll Number	24011101126	
Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

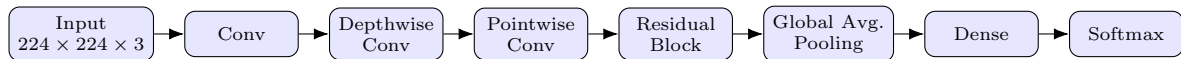
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Students should identify which initialization gives faster and more stable convergence.

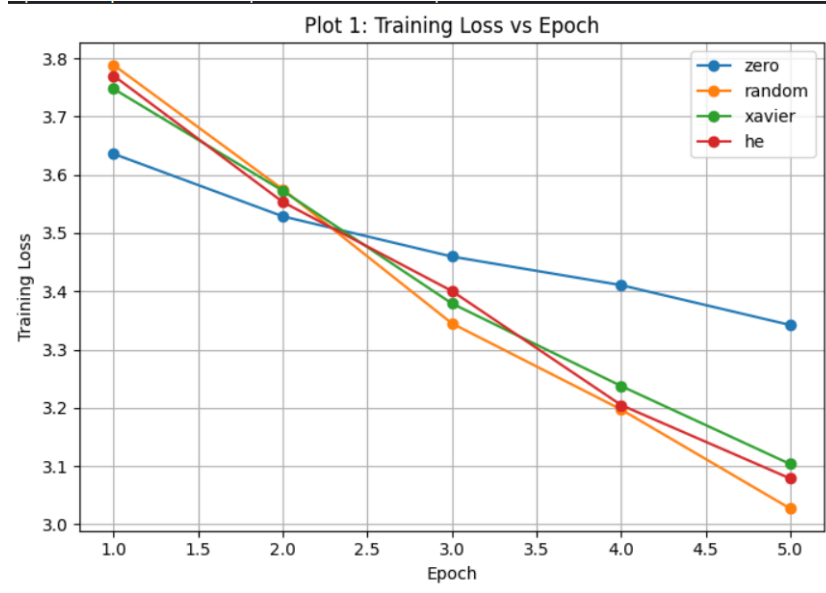


Figure 1: Plot 1: Training Loss vs. Epoch for different weight initialization strategies.

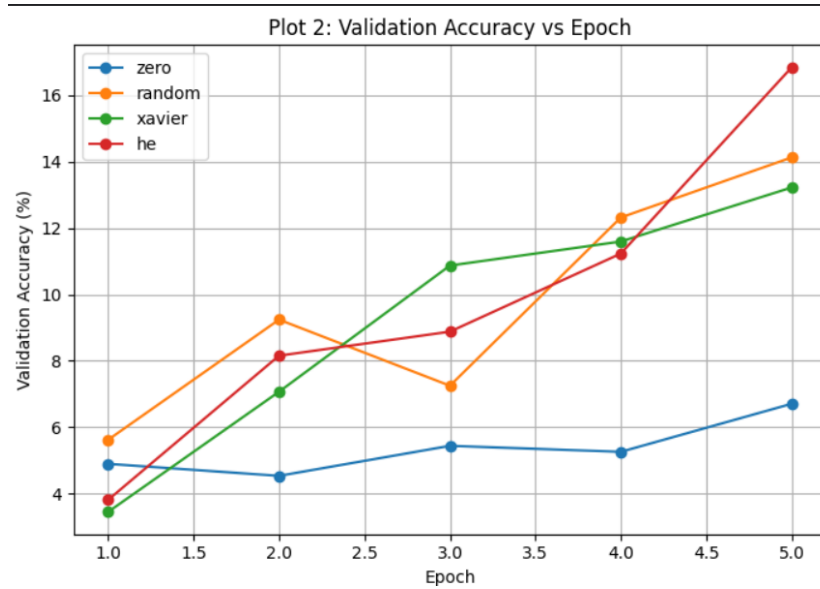


Figure 2: Plot 2: Validation Accuracy vs. Epoch for different weight initialization strategies.

Observed Results (5 epochs, MobileNetV2 trained from scratch):

Initialization	Final Train Loss	Best Val. Accuracy
Zero	3.3421	6.70%
Random	3.0270	14.13%
Xavier/Glorot	3.1033	13.22%
He	3.0785	16.85%

Inference: Zero weight initialization performed the worst, which is expected because all neurons start with the same weights and are unable to learn different features properly. Random, Xavier, and He initialization were able to converge, but He initialization performed the best overall, with faster loss reduction and the highest validation accuracy among the four methods . **Best initialization: He.**

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Students should identify the generalization gap.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Students should identify whether validation loss begins to increase while training loss continues to decrease.

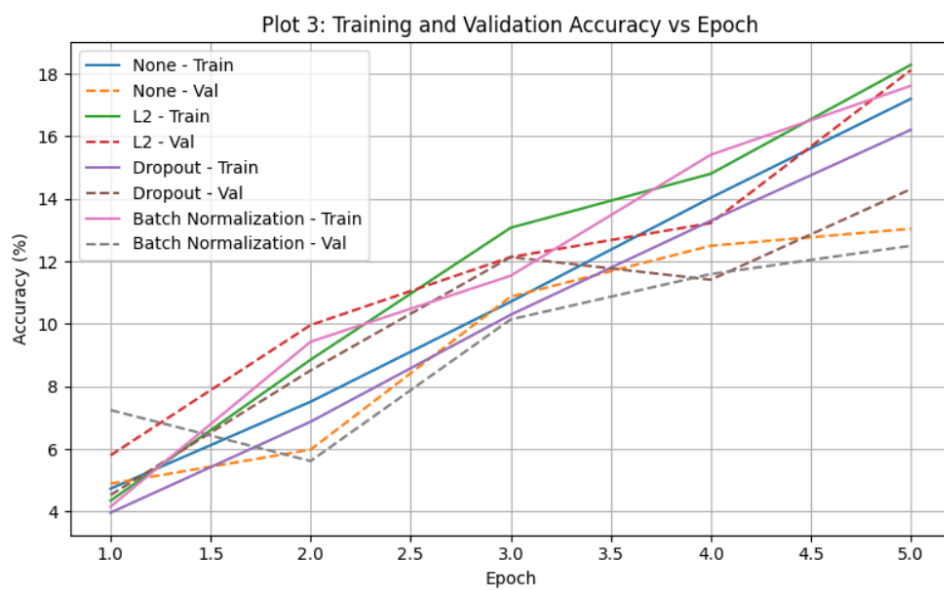


Figure 3: Plot 3: Training and Validation Accuracy vs. Epoch for each regularization strategy.

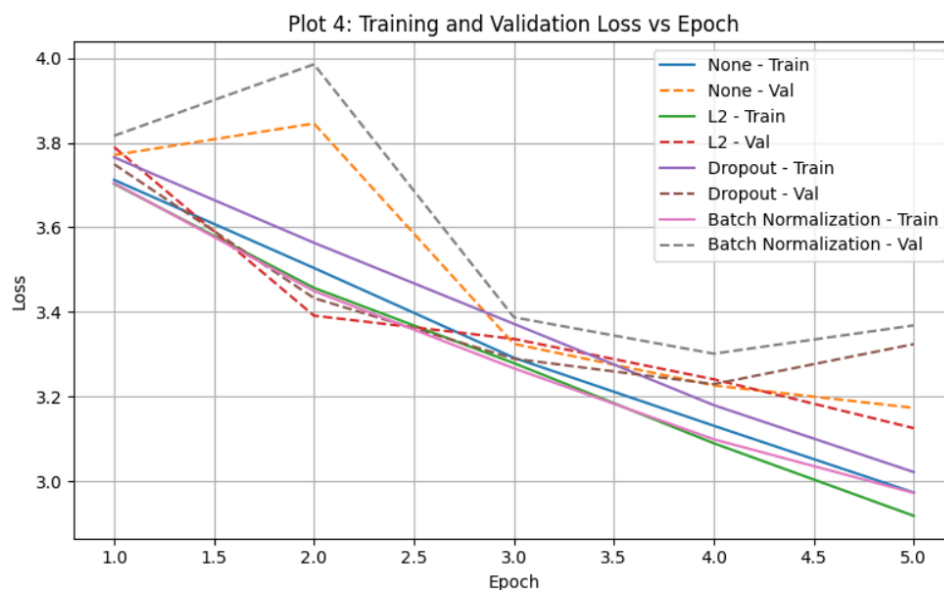


Figure 4: Plot 4: Training and Validation Loss vs. Epoch for each regularization strategy.

Observed Results (He initialization, Adam optimizer, 5 epochs):

Regularization	Final Train Loss	Best Val. Accuracy
None	2.9731	13.04%
L2	2.9181	18.12%
Dropout	3.0216	14.31%
Batch Normalization	2.9722	12.50%

Inference: All four settings still show training accuracy increasing faster than validation accuracy, indicating a generalization gap that would widen with more epochs. L2 regularization gives the best validation accuracy at this training budget, mildly outperforming plain Dropout, while the model already contains Batch Normalization layers by default so the “Batch Normalization” run behaves similarly to the unregularized baseline. **Best regularization: L2.**

7. Batch Normalization

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

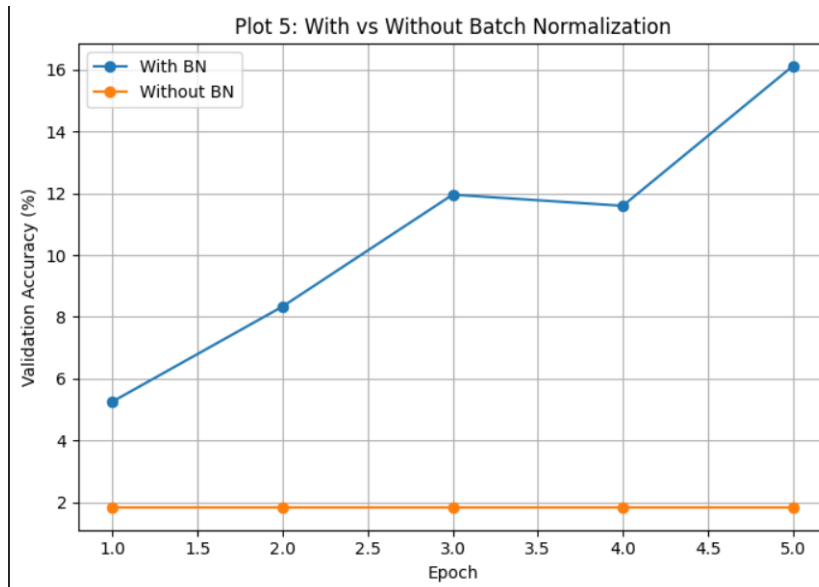


Figure 5: Plot 5: Validation Accuracy vs. Epoch, with and without Batch Normalization.

Observed Results: With BatchNorm layers present, validation accuracy improves steadily to 16.12% by epoch 5. When all BatchNorm layers are replaced by identity layers, training stalls completely, with the loss staying near 3.61 and validation accuracy remaining at 1.81% throughout.

Inference: Removing Batch Normalization caused the model to perform very poorly. The loss barely changed and the validation accuracy stayed close to random guessing. This shows that Batch Normalization is important for keeping training stable and allowing the model to learn properly.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

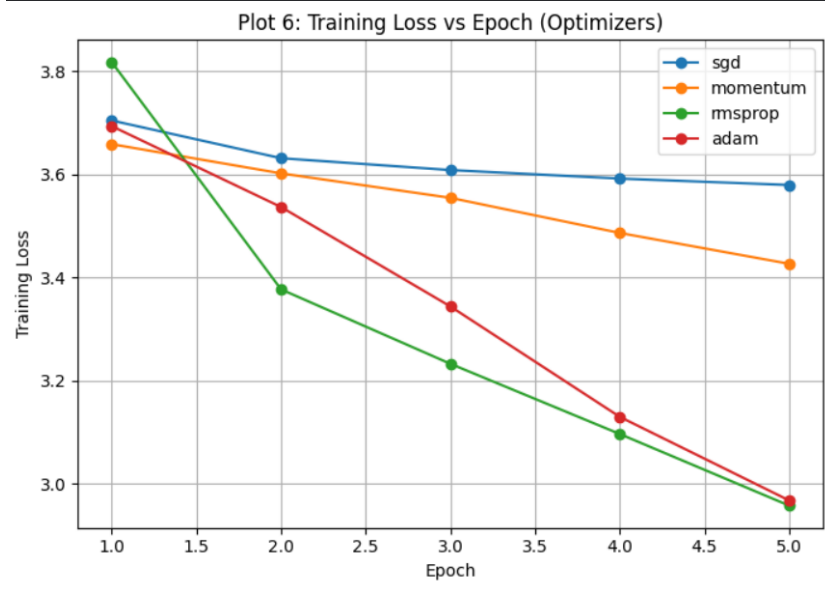


Figure 6: Plot 6: Training Loss vs. Epoch for different optimizers.

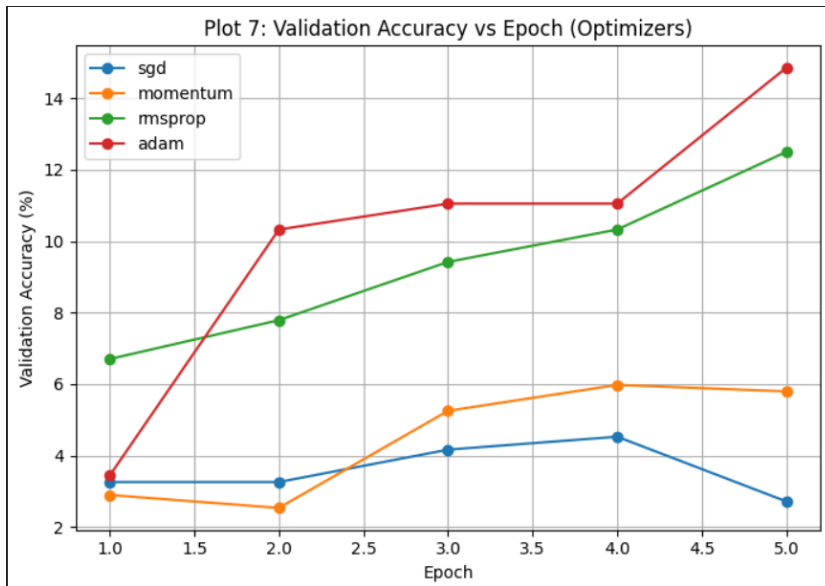


Figure 7: Plot 7: Validation Accuracy vs. Epoch for different optimizers.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	3.5798	4.53%	4	75.09 s
Momentum	3.4269	5.98%	4	75.00 s
RMSPProp	2.9578	12.50%	5	75.16 s
Adam	2.9678	14.86%	5	75.33 s

Inference: Plain SGD and Momentum converge very slowly within 5 epochs and remain close to the random-guess baseline ($1/37 \approx 2.7\%$). Adaptive methods (RMSPProp, Adam) reduce training loss much faster and reach substantially higher validation accuracy in the same number of epochs, with Adam giving the best result. **Best optimizer: Adam.**

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

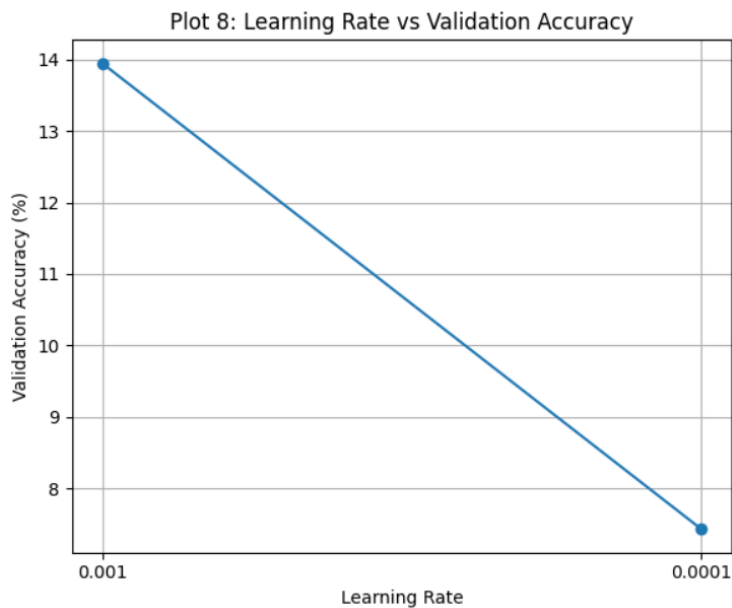


Figure 8: Plot 8: Learning Rate vs. Validation Accuracy.

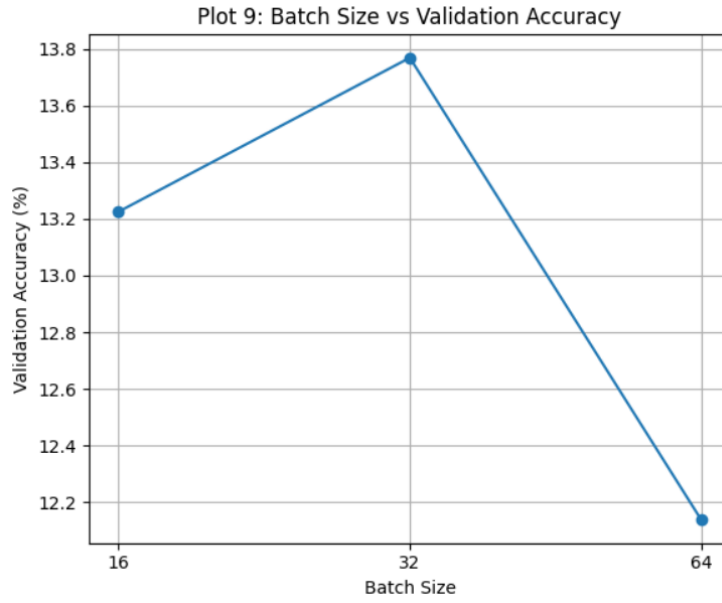


Figure 9: Plot 9: Batch Size vs. Validation Accuracy.

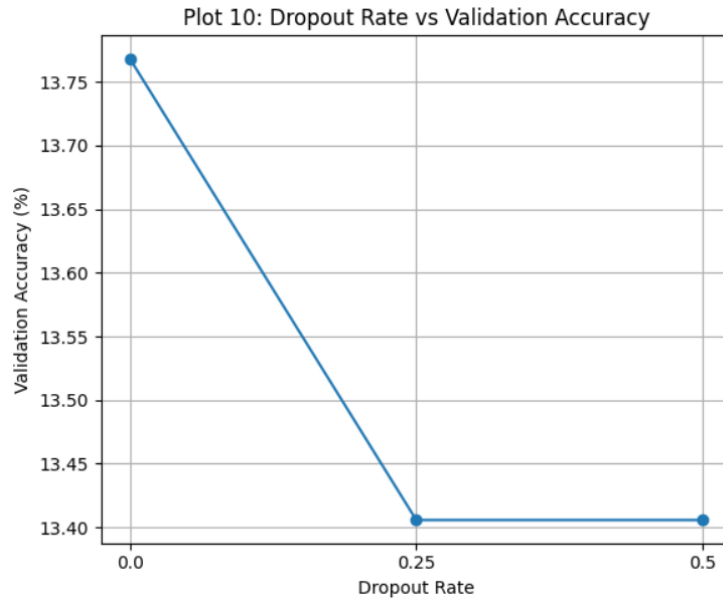


Figure 10: Plot 10: Dropout Rate vs. Validation Accuracy.

Observed Results (best val. accuracy over 5 epochs, other settings held at defaults $lr=0.001$, $batch=32$, $dropout=0.25$):

Setting	Best Val. Accuracy
LR = 0.001	13.95%
LR = 0.0001	7.43%
Batch = 16	13.22%
Batch = 32	13.77%
Batch = 64	12.14%
Dropout = 0.0	13.77%
Dropout = 0.25	13.41%
Dropout = 0.5	13.41%

Inference: A larger learning rate (0.001) clearly outperforms the smaller one (0.0001) within a 5-epoch budget, since the smaller rate has not yet converged. Batch size 32 performs marginally better than 16 or 64. Increasing

dropout does not help at this short training budget, since the model is still underfitting rather than overfitting.
Best hyperparameters: lr = 0.001, batch size = 32, dropout = 0.0.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Students should compare the loss curves before and after fine-tuning.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

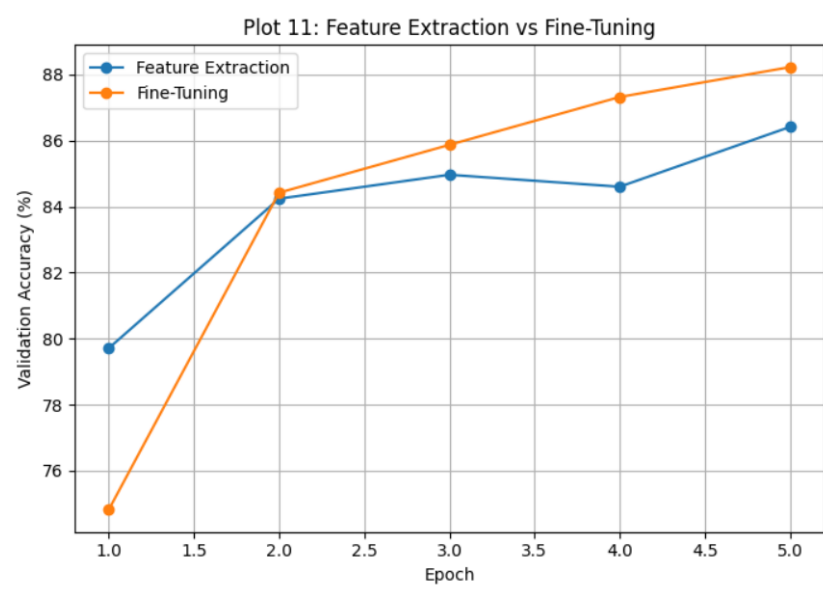


Figure 11: Plot 11: Feature Extraction vs. Fine-Tuning – Validation Accuracy vs. Epoch.

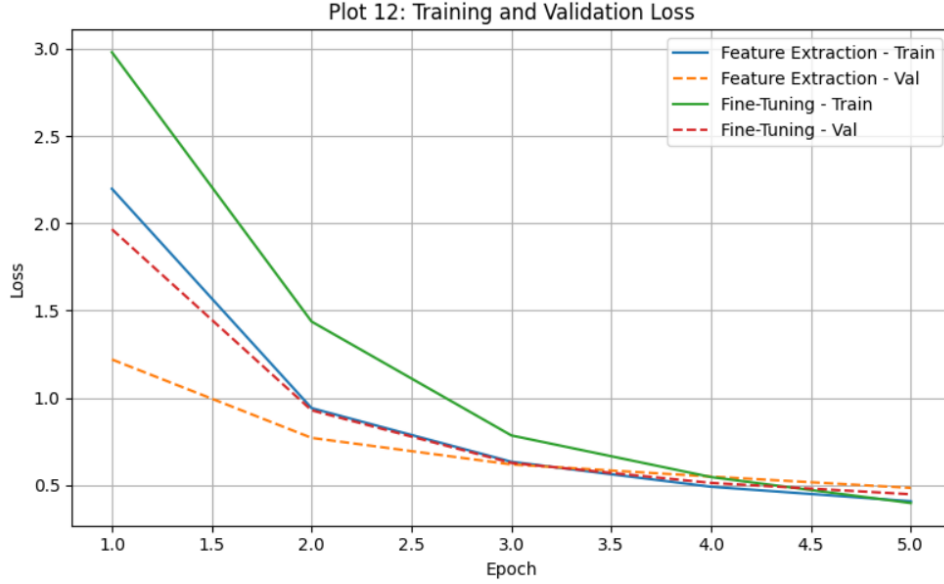


Figure 12: Plot 12: Training and Validation Loss, Feature Extraction vs. Fine-Tuning.

Observed Results (ImageNet-pretrained MobileNetV2, 5 epochs):

Strategy	Final Train Acc.	Best Val. Accuracy
Feature Extraction (frozen base)	92.62%	86.41%
Fine-Tuning (last 4 layers unfrozen, lr = 10^{-4})	92.04%	88.22%

Inference: Using ImageNet-pretrained weights gives a better improvement compared with training MobileNetV2 from scratch. This shows that transfer learning works well for a dataset of this size. Fine-tuning the last few layers with a small learning rate improves the accuracy even further, while feature extraction is still useful and is less likely to affect the pretrained features.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Four configurations were evaluated: C1 (lr = 0.001, batch = 32, dropout = 0.0 – the best individually-tuned hyperparameters), C2 (lr = 0.001, batch = 32, dropout = 0.25), C3 (lr = 0.0001, batch = 32, dropout = 0.25) and C4 (lr = 0.001, batch = 16, dropout = 0.0).

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1	11.34	13.42	13.58	11.84	15.04	13.04 $\pm$ 1.32
C2	11.18	12.46	13.58	12.80	13.12	12.63 $\pm$ 0.81
C3	8.79	6.23	5.27	7.04	6.72	6.81 $\pm$ 1.15
C4	12.14	14.22	14.06	13.60	13.76	13.56 $\pm$ 0.74

(All values are validation accuracy in %.)

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

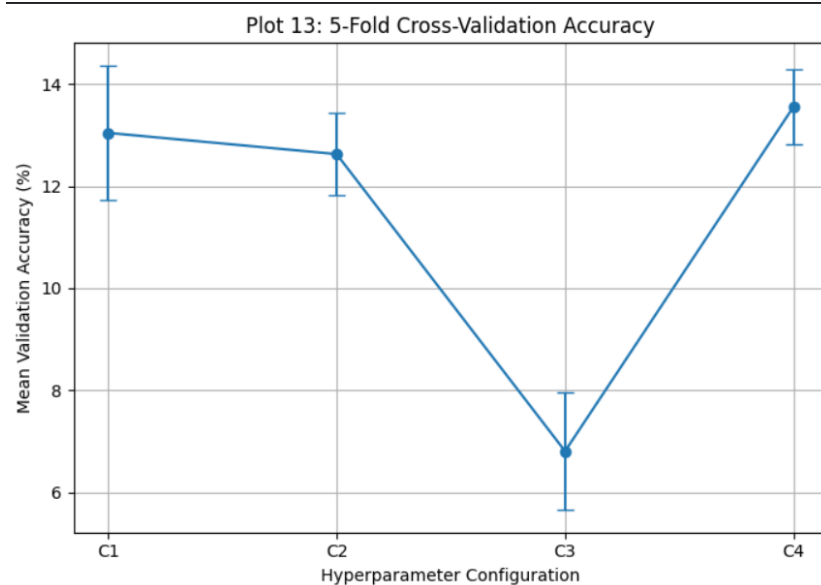


Figure 13: Plot 13: 5-Fold Cross-Validation mean accuracy with standard-deviation error bars for each configuration.

Students should discuss both mean performance and variability.

Inference: C3 (the smaller learning rate) is clearly the weakest configuration and can be ruled out. Among the remaining three, C4 (batch size 16) gives both the highest mean accuracy (13.56%) and the lowest variability (SD = 0.74), making it the most consistent choice, closely followed by C1. **Best CV configuration: C4.**

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	13.56%
CV Standard Deviation	0.74%
Test Accuracy	14.88%
Precision (weighted)	13.25%
Recall (weighted)	14.88%
F1-score (weighted)	10.90%
Training Time	77.80 s
Number of Parameters	2,271,269

Plot 14: Confusion Matrix

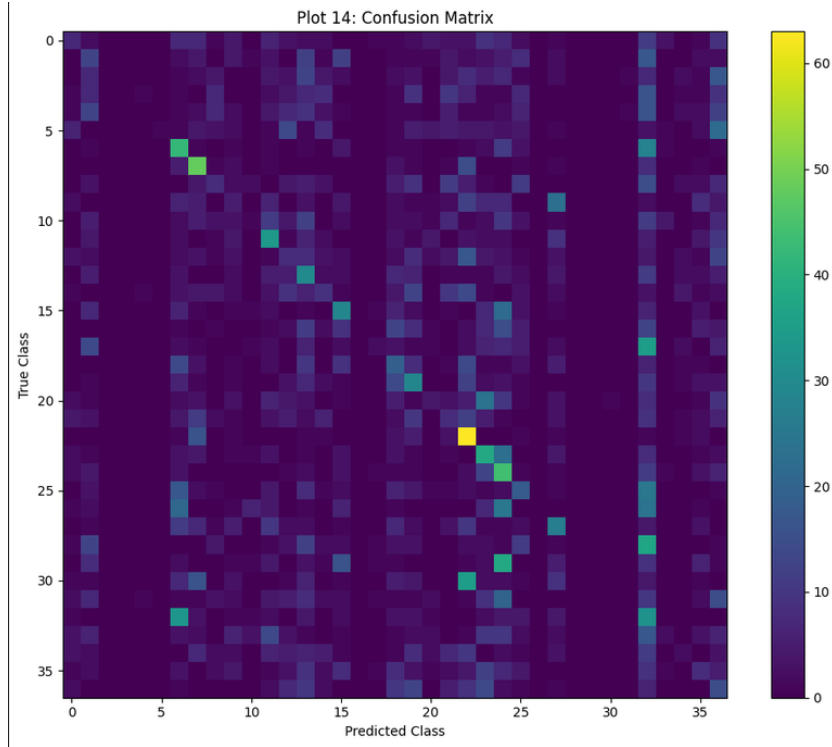


Figure 14: Plot 14: Confusion matrix of the final selected model (C4) on the independent test set.

Inference: The final test accuracy (14.88%) is consistent with the CV mean (13.56%), confirming that cross-validation gave a reliable estimate of generalization on this held-out data. The low weighted F1-score relative to accuracy suggests predictions are concentrated on a subset of breeds rather than spread evenly across all 37 classes – this is expected because the final model here is trained from scratch (unlike the pretrained model in Section 10) and has only had a 5-epoch budget, so it has not yet learned fine-grained, breed-distinguishing features for every class.

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.

13. Overall Results

Configuration	Accuracy	SD	Evaluation	Time
Baseline (no reg., Adam)	13.04%	–	Validation	–
Best Initialization (He)	16.85%	–	Validation	–
Best Regularization (L2)	18.12%	–	Validation	–
Best Optimizer (Adam)	14.86%	–	Validation	75.33 s
Best Hyperparameters (C4)	13.56%	0.74%	5-Fold CV	77.80 s
Fine-Tuned Model (Pretrained)	88.22%	–	Validation	–

Final Model Justification: The fine-tuned pretrained model performed the best, achieving a validation accuracy of 88.22%, which is much higher than the models trained from scratch. The best hyperparameter model achieved 13.56% mean 5-fold cross-validation accuracy with an SD of 0.74%, showing consistent performance across the folds. Therefore, the fine-tuned pretrained model was selected as the final model because it gave the best overall performance and generalization.

14. Questions

What is the difference between model parameters and hyperparameters?

Parameters are values like weights and biases that the model learns during training. Hyperparameters such as learning rate, batch size, dropout and optimizer are chosen before training and control how the model learns.

Why is weight initialization important?

Weight initialization decides the starting point of the model. A poor initialization can make training very slow or unstable, while a good one helps the model converge faster and keeps the activations and gradients stable.

Why can zero initialization be problematic for neural networks?

With zero initialization, all neurons start with the same weights and receive the same gradients. Because of this, they keep learning the same thing instead of learning different features. This is why zero initialization performed the worst in the experiment.

Compare Xavier and He initialization.

Xavier initialization is generally suited to activations like tanh and sigmoid, while He initialization works better with ReLU-based activations. Since MobileNetV2 uses ReLU6, He initialization is a better fit. This was also seen in the results, where He achieved 16.85% validation accuracy compared to 13.22% with Xavier.

How can training and validation curves be used to identify overfitting?

Overfitting happens when training accuracy keeps increasing but validation accuracy stops improving or starts falling. Similarly, training loss keeps decreasing while validation loss starts increasing. A large gap between the two usually indicates poor generalization.

How does Dropout reduce overfitting?

Dropout randomly turns off some neurons during training. This prevents the model from depending too much on specific neurons and helps it learn more general features, which improves generalization.

What is the purpose of Batch Normalization?

Batch Normalization keeps the activations more stable during training by normalizing them within each batch. This helps the model converge faster and makes training more stable. In our experiment, removing BatchNorm caused training to stall.

Explain the numerical Batch Normalization example.

For $x = [2, 4, 6, 8]$, the mean is 5 and the variance is 5. After normalization, the values become approximately $[-1.342, -0.447, 0.447, 1.342]$. With $\gamma = 1$ and $\beta = 0$, these remain the final outputs of the Batch Normalization layer.

What are the roles of γ and β ?

γ controls the scale of the normalized values, while β controls their shift. Both are learned during training and allow the model to adjust the normalized output when needed.

Compare SGD, Momentum, RMSProp and Adam.

SGD updates the weights directly using the current gradient. Momentum uses previous gradients to make updates smoother and faster. RMSProp adapts the learning rate for each parameter, while Adam combines the ideas of Momentum and RMSProp. In our experiment, Adam gave the best validation accuracy.

What happens when the learning rate is too large?

The model can overshoot the optimal solution, causing the loss to oscillate or even increase. This can make training unstable and reduce validation accuracy.

What happens when the learning rate is too small?

Training becomes very slow because the weight updates are too small. The model may need many more epochs to reach a good solution.

What is the effect of increasing batch size?

A larger batch gives more stable gradient estimates but requires more memory and results in fewer updates per epoch. A smaller batch adds more noise to the gradients, which can sometimes help generalization. In our experiment, batch size 32 gave the best result.

Explain stride and padding.

Stride is how far the filter moves across the image each time. A larger stride reduces the output size. Padding adds extra pixels around the image and helps control the output size while allowing the filter to process the border properly.

Why is MobileNetV2 computationally efficient?

MobileNetV2 uses depthwise separable convolutions and inverted residual blocks, which reduce the number of parameters and computations compared to standard CNNs. This makes it much lighter while still giving good performance.

What is depthwise separable convolution?

It splits a normal convolution into two steps: a depthwise convolution that works separately on each channel and a pointwise 1×1 convolution that combines the channels. This greatly reduces the amount of computation.

What is transfer learning?

Transfer learning means using a model that was already trained on a large dataset, such as ImageNet, and adapting it to a new task. Instead of learning everything from scratch, the model can reuse features like edges, textures and shapes.

Differentiate feature extraction and fine-tuning.

In feature extraction, the pretrained base is kept frozen and only the new classifier is trained. In fine-tuning, some of the pretrained layers are also unfrozen and trained with a smaller learning rate. In our experiment, feature extraction reached 86.41% validation accuracy, while fine-tuning improved it to 88.22%.

Why is a smaller learning rate generally used during fine-tuning?

The pretrained model already has useful features, so a large learning rate could change the weights too much and damage what the model has already learned. A smaller learning rate allows the model to adapt without losing those useful features.

Why is K-Fold Cross-Validation useful for hyperparameter selection?

A single train-validation split can give a result that depends heavily on which samples happen to be in the validation set. K-Fold CV uses multiple splits and gives both an average performance and its variability, making the comparison between configurations more reliable.

Why must the test set remain untouched during tuning?

The test set should only be used for the final evaluation. Using it during tuning would give the model information about the test data and could make the final accuracy look better than it really is.

Why should mean and standard deviation both be reported?

The mean shows the average performance, while the standard deviation shows how much the performance changes across folds. Reporting both gives a better idea of how accurate and consistent the model actually is.

Is the highest validation accuracy always sufficient to select a model?

No. A model can get the highest validation accuracy just because of a favourable split. Model selection should also consider cross-validation results, variability, generalization and computational cost.

15. Conclusion

The experiments showed that different training choices can have a major impact on CNN performance. Among the models trained from scratch, He initialization, Batch Normalization, Adam optimizer, and suitable hyperparameters gave better results than the other configurations. The results also showed that poor choices such as zero initialization or removing Batch Normalization can make training very slow or even stop the model from learning properly.

The biggest improvement came from transfer learning with the pretrained MobileNetV2 model. Feature extraction achieved 86.41% validation accuracy, while fine-tuning the last few layers improved this further to **88.22%**. This clearly shows that using pretrained features is much more effective for this dataset than training the network completely from scratch. The 5-fold cross-validation results also helped compare the configurations more reliably by showing both their average performance and variability.

Overall, the fine-tuned pretrained MobileNetV2 model was selected as the final model because it gave the best accuracy and generalization performance in the experiments.

16. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>

Source Code: <https://github.com/vinukohshawenn/deeplearningexperiments/tree/main/oxford-cnn>